MACHINE LEARNING FOUNDATIONS

Regularization



upna
Universidad Pública de Navarra
Nafarroako Unibertsitate Publikoa

Agenda

01 Recap

02 Weight decay

03 Dropout

04 Early stopping

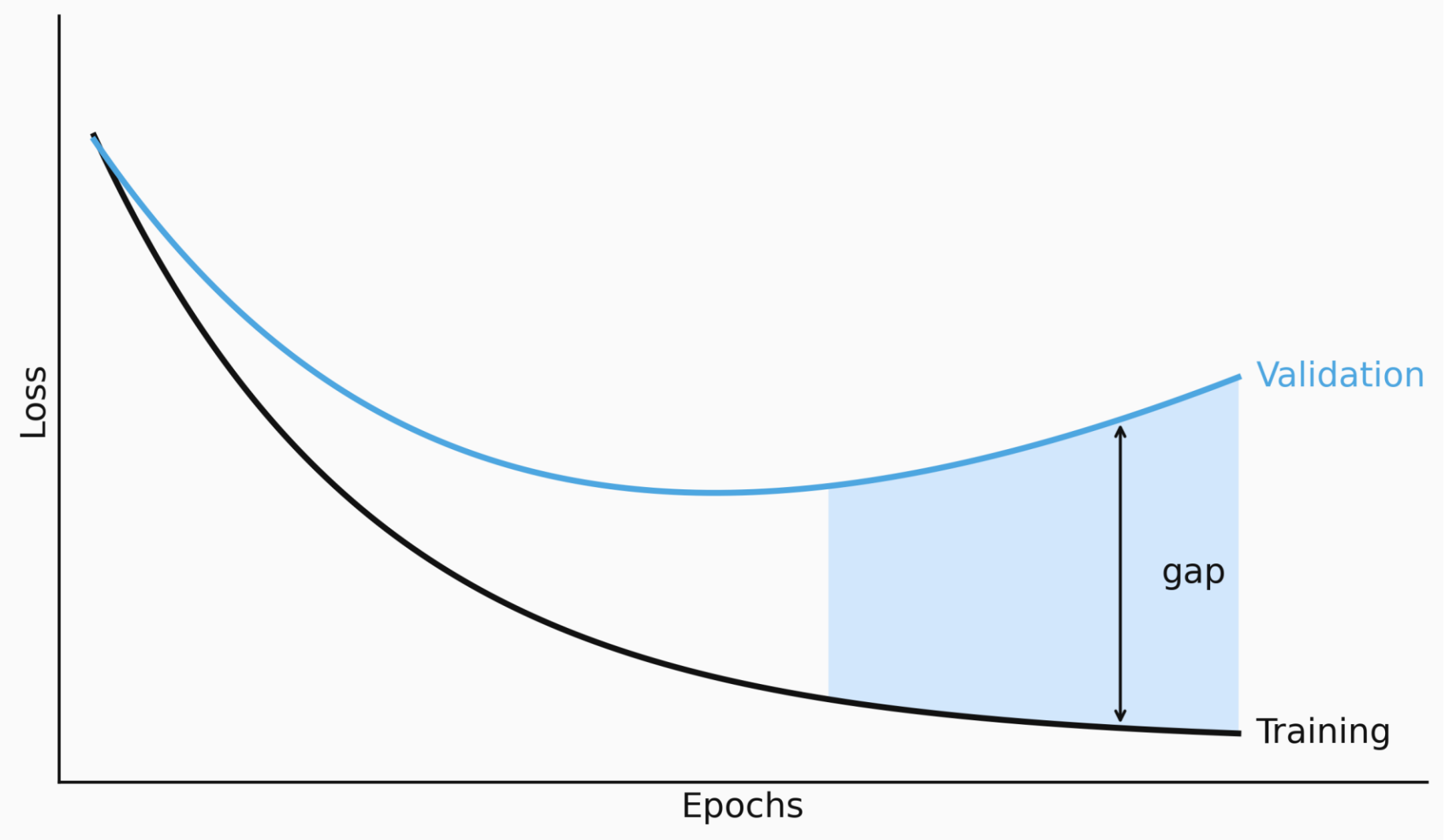
Recap

- MLPs with many hidden layers allow to model very complex problems
- However, when we work with deep neural networks, many problems arise
  - Exploding/vanishing gradients
  - Complex cost functions, difficult to optimize
  - Overfitting
- The generalization capability of models is one of the most importat issues when dealing with neural networks
  - Having more data always is desirable, but hard to achieve
  - We need to incorporate regularization techniques

Recap

- In ML, our approach on training models consists of two phases
  - Fit the training data
  - Estimate the generalization error evaluating the model on holdout data
- The difference between our fit on the training data and on test data is called the generalization gap
  - If it is large, then our model overfits
- Interpretation: our model is too complex and we require
  - Shrink the number of features or the number (and size) of learned parameters

Recap



Classical regularization methods for DL

- The classical method for regularization of ML models is by restricting the values that the parameters can take
  - Rather than reducing the number of parameters
- The most common is ℓ_2 regularization, that imposes a penalty to the cost function which is dependent on the norm of the weights
  - We assume that the function $f = 0$ is the simplest one, and that we can measure the complexity of a function by the distance of its parameters to zero
- Assuming a weighting vector w, the cost function J=L is transformed into

$$J = L + \frac{\lambda}{2}\|w\|^2 = L + \frac{\lambda}{2}\sum_i w_i^2$$

Weight decay

- Observe that, in the backpropagation, if we denote by s the regularization term, we have that

$$\frac{\partial J}{\partial w} = \frac{\partial L}{\partial w} + \frac{\partial s}{\partial w} = \frac{\partial L}{\partial w} + \lambda w$$

- When update w, we have that

$$w = w - \eta \frac{\partial J}{\partial w} = w - \eta \frac{\partial L}{\partial w} - \eta \lambda w = w(1 - \eta\lambda) - \eta \frac{\partial L}{\partial w}$$

- Assuming that $\lambda > 0$, we have that prior to update, the values of w are reduced
  - That is why this is called weight decay
- In practice, the regularization term is usually applied only to weights, not to the biases

L1 vs L2 regularization

- Another common penalty is L1 regularization, which uses the sum of absolute values instead of the squared norm
  - $J = L + \lambda \sum_i |w_i|$
- The two penalties behave differently
  - L2 (weight decay) shrinks all weights smoothly towards zero, but rarely makes them exactly zero
  - L1 tends to push some weights exactly to zero, producing sparse solutions (some kind of feature selection)
- Geometric intuition
  - L2 constraints weights to a sphere and L1 to a diamond
- In DL, L2 is used more often

Regularization via noise injection

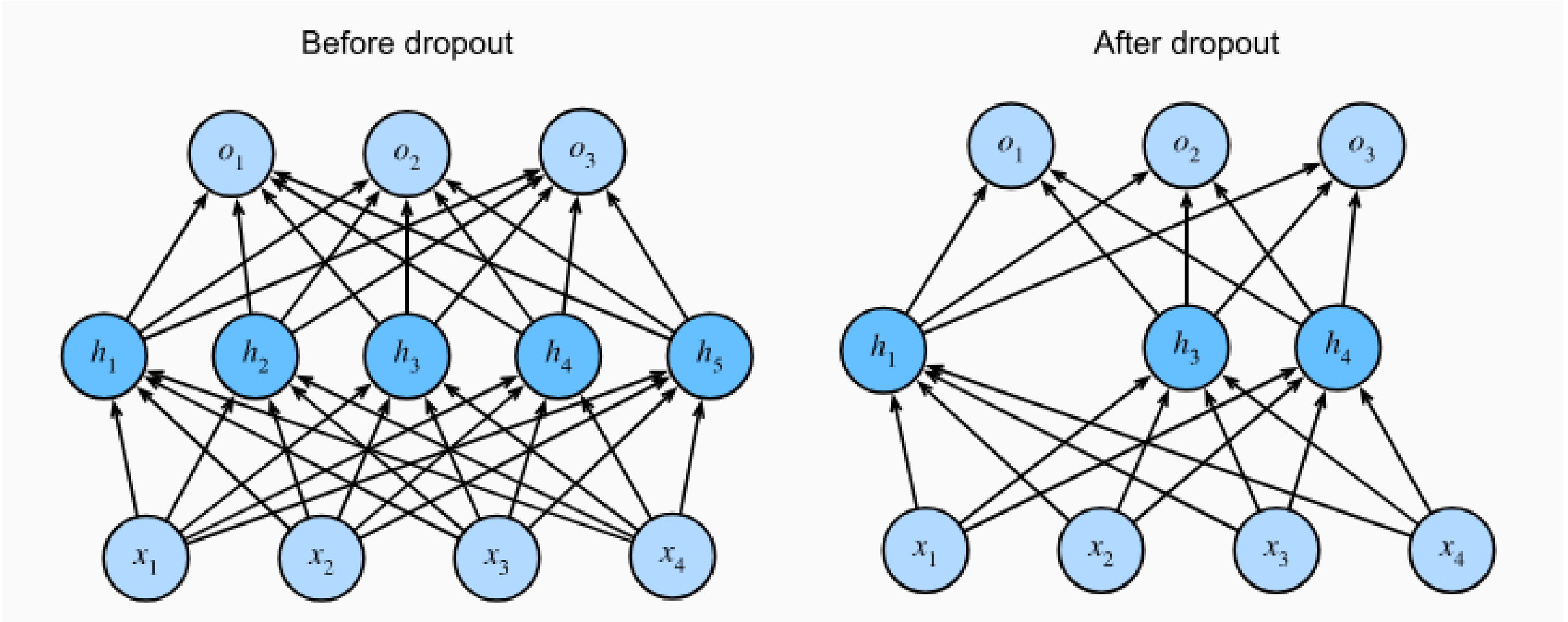
- Regularization can be done in several ways
  - Reducing number of dimensions/parameters to the model
  - Reducing the norm of parameters (weight decay)
  - Training more smooth functions (should not be sensitive to small changes to its inputs)
- Tha latter can be implemented by injecting noise either to the inputs or to the layers of a NN

Dropout

- Dropout regularization involves injecting noise while computing each internal layer during forward propagation
  - It literally drops out some neurons during training, zeroing out some fraction of the neurons in each layer before calculating the subsequent layer
- However, injection should be done in an unbiased manner
  - Expected value of each layer should be equal to the value it would have taken without noise
- Formally, if we apply dropout with probability p to an intermediate activation h, its activation is replaced by a random variable

$$h' = \begin{cases} 0 & with\ probability\ p \\ \dfrac{h}{1-p} & otherwise \end{cases}$$

Dropout



Dropout

- Why does it work?
  - Each training step effectively trains a different, “thinned” sub-network (a different subset of active neurons)
  - This can be seen as training an implicit ensemble of sub-networks that share weights, and averaging them at test time
  - It also prevents neurons from co-adapting: since a neuron cannot rely on any specific other neuron being present, it is forced to learn more robust features on its own
- It is important to note that, during test time, we usually disable dropout
  - This is done by calling model.eval()

Early stopping

- Finally, early stopping is a another well-known regularization technique that can be used together with weight decay, dropout, etc.
- As mentioned before, deep NN can keep reducing the training error as we add more iterations (potentially until memorizing the training set)
  - This means that if we limit the number of iterations, we can have some regularization effect
- The most common way to determine stopping criterion is to monitor validation error throughout training
  - Typically by checking after each epoch
  - We cut off training if validation error has not decreased by more than ϵ for some number of epochs (patience criterion)
- Especially useful with label noise or intrinsic variability in the label

Early stopping

